Smart Factory Equipment Anomaly Alert Agent

Pegatron ML Engineer Assignment 3

Vaibhav Kumar Sunkaria

github.com/vaibhavsunkaria97/smart-factory-agent

Contents

Problem and objective

Assignment 3 — smart factory equipment anomaly alerting

- Equipment degrades gradually. Temperature, pressure and vibration drift before a failure occurs.
- An operator requires three things from an alert: which machine, how urgent, and what action to take.
- Alert volume is itself a failure mode. Excessive alerting causes alarm fatigue, and genuine faults are then ignored.

Objective	Constraint
Detect anomalies in equipment sensor data and emit prioritised, actionable alerts in real time.	Free AI tools only. Highest achievable level of automation. Minimum usage barrier for the end user.

System architecture

Perceive ⟶ reason ⟶ act, with graceful degradation at the reasoning layer

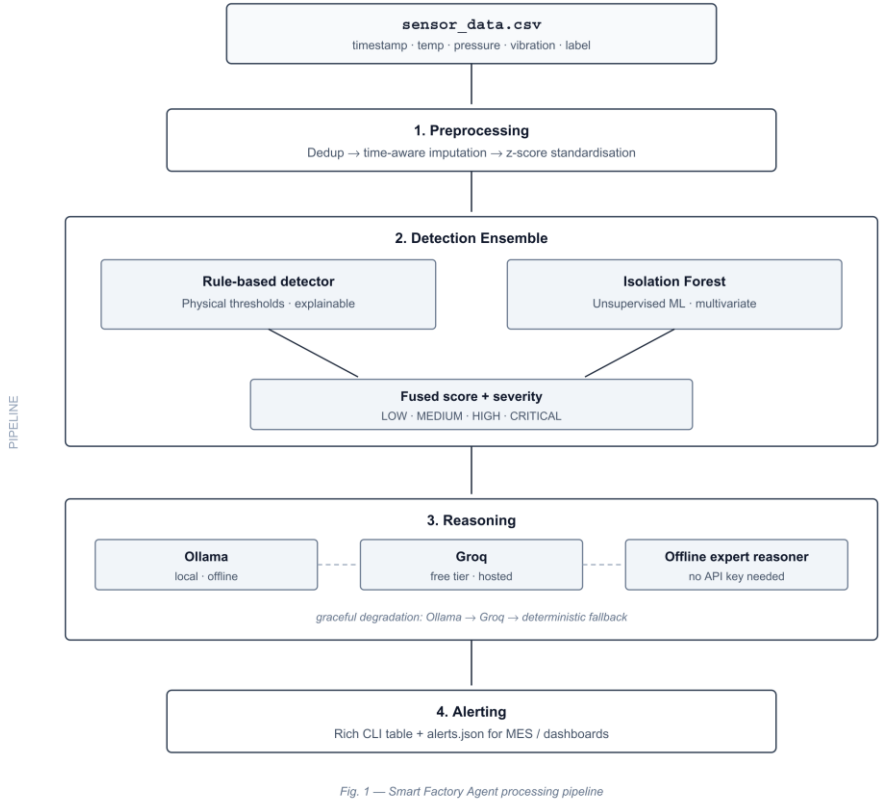


Figure 1 — Four-stage pipeline: preprocessing, detection ensemble, reasoning, alerting.

Detection design: two complementary detectors

Precision and explainability, combined with recall on unknown fault modes

	Rule-based detector	Isolation Forest
Basis	Physical thresholds from the specification	Unsupervised multivariate density
Detects	Faults that can be described in advance	Combinations no single threshold expresses
Training data	None required — operational from day one	Learns the normal operating envelope
Limitation	Blind to any fault inside the limits	Lower interpretability

Scores are fused as an OR-of-evidence: score = max(rule, ML).

This favours recall, on the basis that a missed fault carries greater cost than a false alarm.

Reasoning layer

Pluggable backends with a deterministic floor

- Each detection is converted into a diagnosis and a recommended action.
- Backends are attempted in priority order: Ollama (local) → Groq (free tier) → deterministic knowledge base.
- Any failure — network, timeout, malformed response — degrades to the deterministic path without interrupting execution.
- The knowledge base is keyed on (signal, direction) and returns genuine maintenance guidance rather than placeholder text.

The LLM never alters a detection. It phrases the explanation only,

so classification remains deterministic and unit-testable.

Use of free AI tools

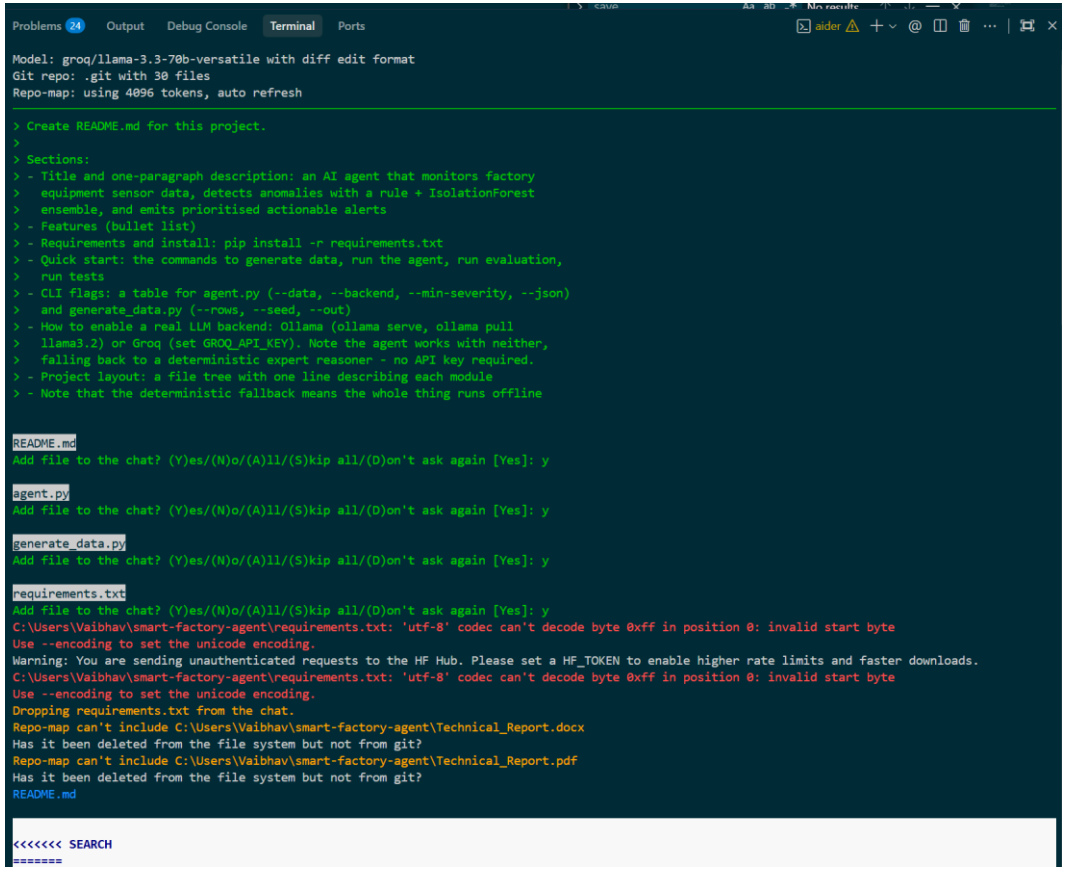
Only free tiers were used at any stage

Tool	Tier	Role	Contribution
Aider + Groq llama-3.3-70b (later Nemotron 3 Ultra)	Free	Build-time	Agentic loop: repository context, change planning, file generation, automatic git commits
Claude	Free	Build-time	Requirement analysis, architecture decisions, defect diagnosis, document generation
Ollama / llama3.2	Free, local	Runtime	LLM embedded in the delivered product, generating each alert's diagnosis
Groq	Free	Runtime	Hosted alternative reasoning backend

- AI is applied in two distinct roles: tools that built the system, and an LLM operating inside the delivered product.
- Model selection is task-dependent: a small local model at runtime for privacy and offline operation; a large free-tier model at build time for code quality.

Agentic development loop in operation

The agent reads the repository, plans the change, writes the files and commits



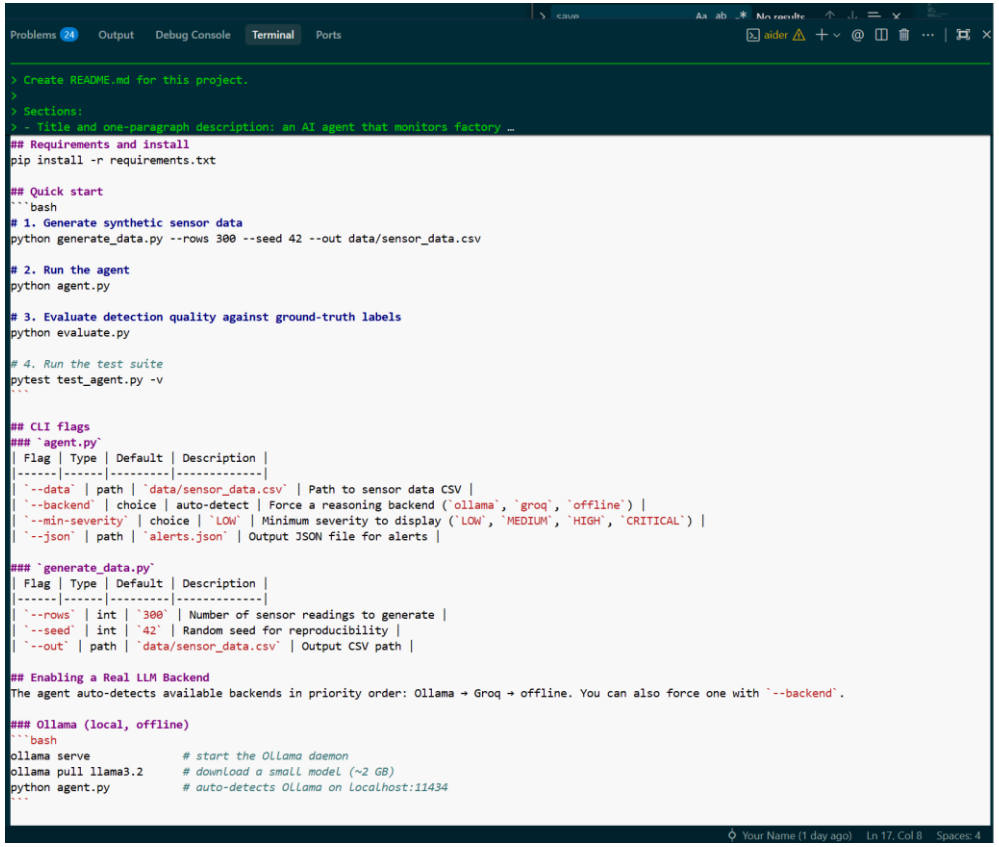


Figure 2 — Left: Aider on groq/llama-3.3-70b with a 30-file repository map. Right: the generated README.

AI as reviewer, not only generator

Every AI-generated module was executed and inspected — five defects were identified

Defect introduced by the AI	How it was identified	Resolution
8 of 41 rows labelled abnormal breached no threshold — recall ceiling of ~80%	Verification script run against the generated dataset	Noise applied after channel corruption; generator-side assertion added
Duplicated loop header and incomplete assertion — file would not parse	Syntax validated prior to execution	Edit re-scoped into smaller changes
Interpolation applied to a string column, raising TypeError	Executed; the statement is valid in isolation	Restricted to numeric sensor columns
Zero missing values injected at 100 rows — imputation became inert	Unit test ran the generator at a size never run manually	Minimum floor applied; injection restricted to normal rows
LLM backend reported success while every call failed	Identified from output quality, not an error condition	Incorrect endpoint; a bare except: pass suppressed the exceptions

The status line reported “ollama” while zero LLM calls had succeeded. No crash, no warning.

Measured limitations of the local model

36 detections processed through both llama3.2 and the deterministic knowledge base

Failure mode	Observed output
Incomplete diagnosis	On rows breaching multiple thresholds, only one signal was addressed
Directional inversion	“Increase operating temperature” recommended for an under-temperature breach
Hallucinated hardware	“Replace sensor 3” — no such sensor exists in the system

The deterministic knowledge base exhibited none of these, as its recommendations are retrieved against detected faults rather than generated.

Execution with a live LLM backend

Cleaning summary, prioritised alert table, and top-priority alert card (python agent.py –min-severity CRITICAL)

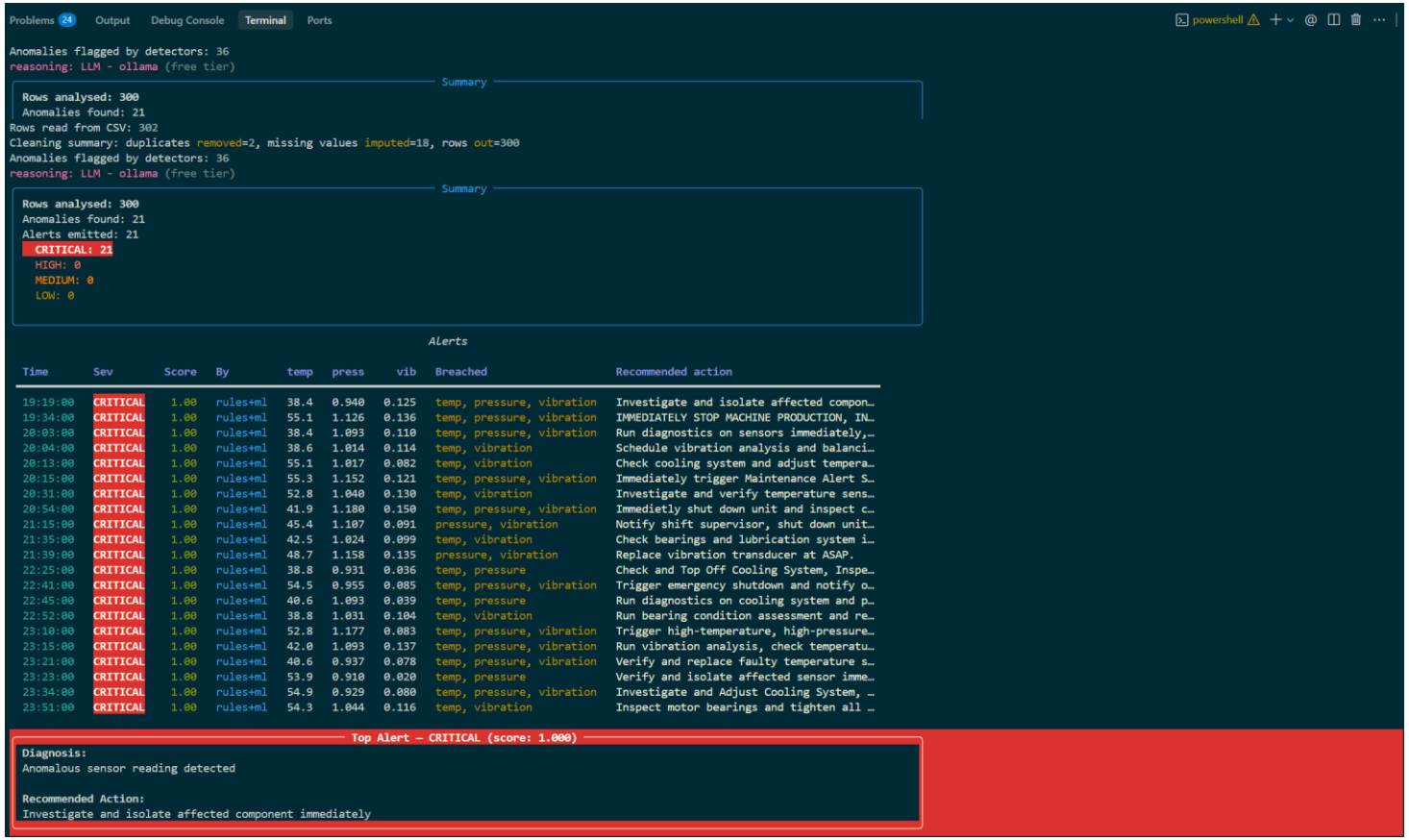


Figure 3 — python agent.py --min-severity CRITICAL. Header confirms reasoning: LLM - ollama.

Execution with no LLM available

Deterministic fallback — the low usage barrier required by the brief

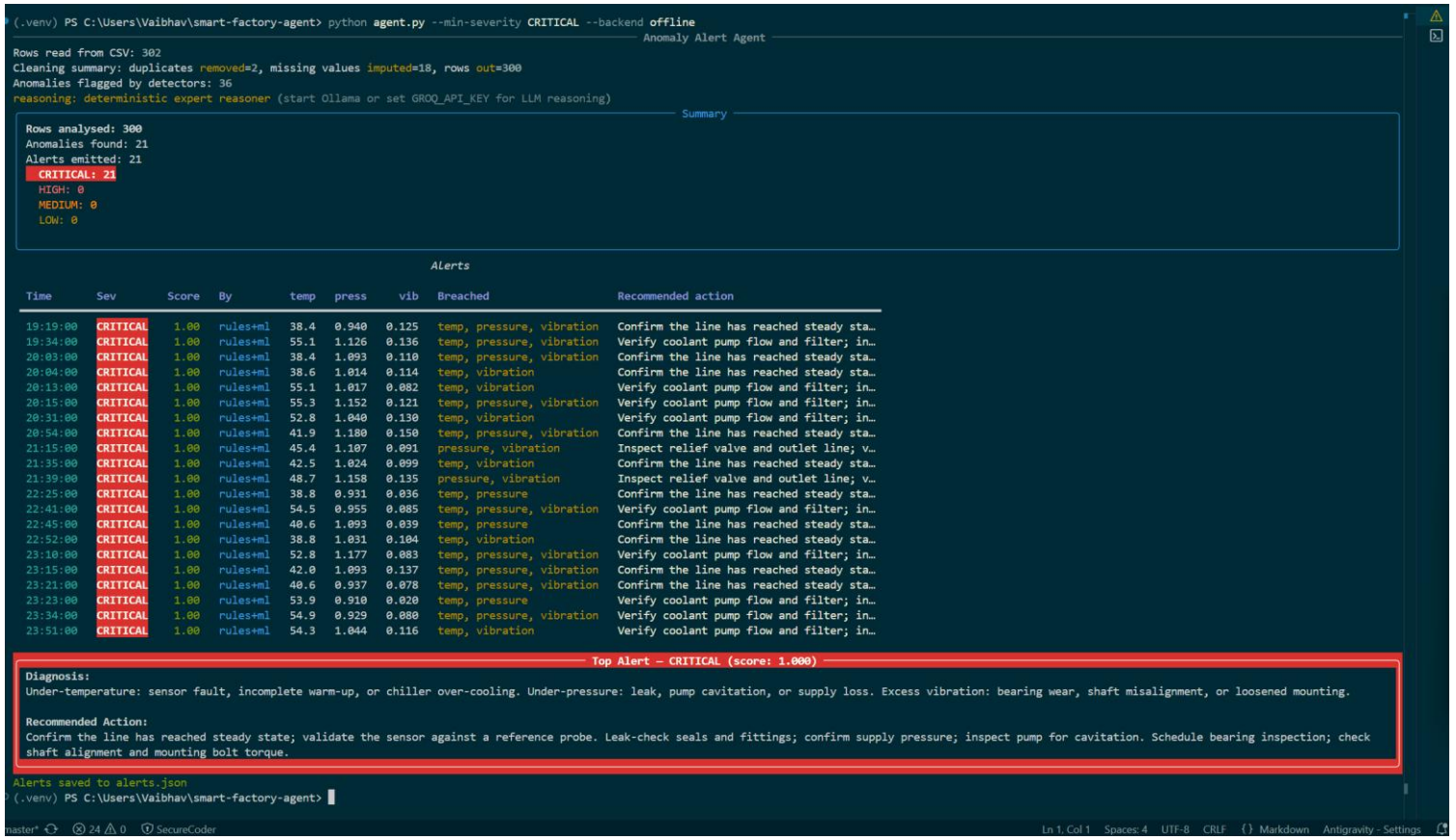


Figure 4 — --backend offline. Identical detections and severities; only the wording differs.

Evaluation results

300 readings · 33 true anomalies · 11 unit tests passing

Detector configuration	Precision	Recall	F1	F2	Flagged
Rule-based only	0.943	1.000	0.971	0.988	35
Isolation Forest only	0.917	1.000	0.957	0.982	36
Gaussian Mixture (K = 2)	0.917	1.000	0.957	0.982	36
Rule + IF ensemble (shipped)	0.917	1.000	0.957	0.982	36

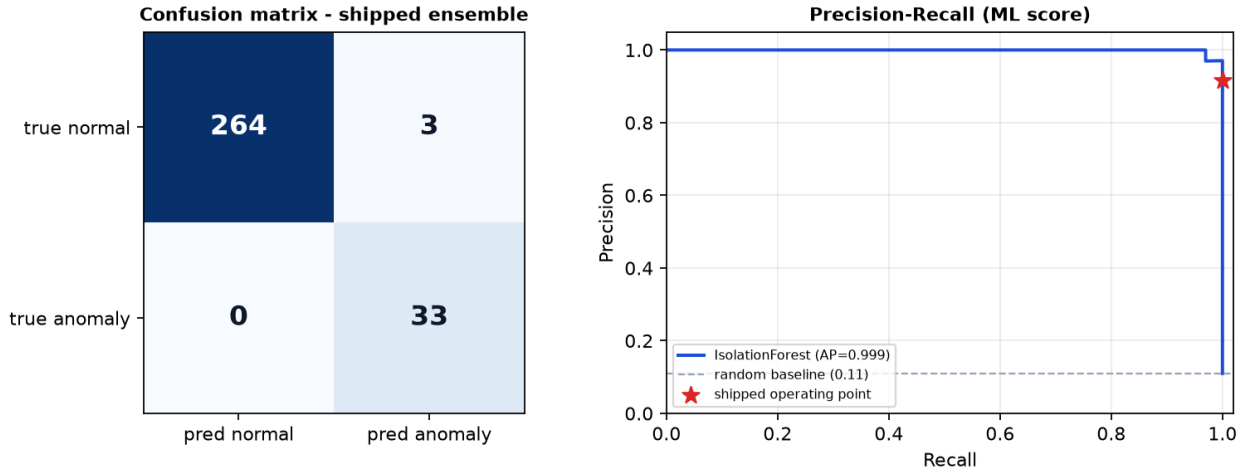


Figure 5 — Confusion matrix and precision–recall curve with the shipped operating point.

Finding: preprocessing manufactured two anomalies

Recall is 1.000. Precision is not, and the cause has been identified

- Two rows contained a missing vibration reading positioned between two high-vibration neighbours.
- Time-aware interpolation filled them with 0.080 and 0.088 — both above the 0.07 threshold.
- Those rows are labelled normal in the source data but breach after cleaning, and are therefore counted as false positives.
- The detectors behaved correctly. The value they were given never occurred.

Cleaning is not a neutral step. Filling a gap can create the fault being searched for.

Limitations

- Point anomalies only. Each reading is scored independently, so gradual drift across a window is not detected.
- A sequence model was deliberately not attempted. The dataset has no temporal correlation; an LSTM would have no learnable structure.
- Imputation is non-causal. Interpolation uses the reading following the gap, which is unavailable in a streaming context.
- Severity thresholds are heuristic. The boundaries were selected to produce a usable distribution, not derived from a cost model.

Next steps

Path from prototype to production deployment

Area	Action
Ingestion	Replace the CSV source with a Kafka consumer; LLM enrichment runs asynchronously off the critical path
Thresholds	Derive operating envelopes per machine from its own history rather than a single global set
Model maintenance	Monitor score distribution and alert rate to trigger retraining before silent degradation
Labelling	Capture operator confirm/dismiss decisions to accumulate genuine fault labels
Recommendations	Retrieve maintenance actions from equipment documentation rather than model priors
Integration	Raise an MES work order automatically on a CRITICAL alert

Thank you

github.com/vaibhavsunkaria97/smart-factory-agent

Vaibhav Kumar Sunkaria